

ITE 1942-ICT PROJECT

PROJECT REPORT LEVEL 01

School Management System

Submitted by:

S. Mathiyazhakan

E2245659

Bachelor of Information Technology (External Degree)

University of Peradeniya

Table of Contents

1. INTRODUCTION-----	1-3
2. RELATED WORKS-----	4-6
3. SYSTEM ANALYSIS-----	7-10
4. SYSTEM DESIGN-----	11-21
5. SYSTEM IMPLEMENTATION-----	22-49
6. APPENDIX-----	50-59
7. REFERENCES-----	60

1. Introduction

1.1. Overview

Technologies are rapidly evolving and playing a major role in various sectors such as airports, bus stations, and hospitals. Nowadays, many schools worldwide are adopting school management systems for efficient and effective administration. However, in Sri Lanka, only a few schools are utilizing such systems.

1.2. Purpose

The purpose of this project is to implement a School Management System to streamline administrative processes in Sri Lankan schools. This system aims to enhance efficiency, accuracy, and communication within schools by leveraging available computer technology beyond teaching IT subjects.

1.3. Background & Motivation

Despite having computer systems, most schools in Sri Lanka use them solely for teaching Information Technology and not for administrative purposes. A few schools use Microsoft Word and Excel for some administrative tasks, but these tools are limited in scope and do not fully utilize the potential of computer systems. There is a significant opportunity to improve school management by automating various administrative tasks

1.4. Problem in Brief

Schools in Sri Lanka face several challenges with their current administrative processes:

- Manual and time-consuming processes for student and teacher enrollment and record-keeping.
- Inefficiencies and errors in data management.
- Difficulty in managing and analyzing large amounts of student data.
- Compromised privacy and confidentiality of personal data.
- Ineffective communication between teachers, students, and administrators.

1.5. Aims & Objectives

The primary aim of this project is to develop a cost-effective School Management System that addresses these challenges. The objectives include:

- Automating administrative processes such as student and teacher enrollment.

- Centralizing record-keeping and data management.
- Ensuring data privacy and confidentiality.
- Improving communication within the school.
- Providing a user-friendly interface for administrators, teachers, and students.

1.6. Summary

In summary, this project seeks to enhance the administrative efficiency of schools in Sri Lanka by implementing a comprehensive School Management System. By automating key processes and improving data management and communication, this system will help schools better utilize their computer resources and overcome existing challenges.

2. Related Works

In this section, we review existing systems similar to our School Management System (SMS) to understand their features, strengths, and weaknesses. This comparison will inform the design and development of our system to ensure it meets the needs of educational institutions effectively. The project seeks to enhance the efficiency and precision of managing information regarding students and teachers, replacing the manual methods that often lead to errors and disruptions in school records. This literature review provides an overview of existing research on automated systems in various fields and their benefits.

Teachers Record Management System Project Report

Acharya (2023) explores a Teachers Record Management System aimed at effectively organizing and managing teacher-related data. The system focuses on functionalities such as personal information management, attendance tracking, and performance evaluations. It highlights the benefits of digital records for enhancing data accuracy, streamlining administrative tasks, and facilitating decision-making processes. Acharya's research emphasizes how centralized data systems can eliminate the redundancies and inaccuracies common in manual record-keeping. The automated system allows for real-time updates and easy retrieval of information, which significantly reduces the workload on administrative staff and ensures that decision-makers have access to accurate and up-to-date information[1].

Online Bus Reservation System

Acharya (2024b) presents an Online Bus Reservation System, focusing on simplifying the booking process for bus travel. The system offers features such as route management, seat selection, and payment processing. The paper highlights the benefits of online reservations, including convenience, time savings, and real-time updates on bus schedules and seat availability, alongside security measures for data protection. This system demonstrates how integrating technology into routine processes can enhance user experience and operational efficiency. By drawing parallels between the bus reservation system and school management tasks, we can see how similar technological solutions can be applied to streamline student enrollment, schedule management, and other administrative functions in schools[2].

Online Shopping Management System

Acharya (2024c) examines the development of an Online Shopping Management System aimed at enhancing user experience in e-commerce. The system's key functionalities include product catalog management, shopping cart integration, and order processing. The study addresses

challenges like user authentication, data privacy, and system scalability, offering insights into optimizing backend processes to handle high transaction volumes. The relevance of this system to school management lies in its ability to handle large datasets and transactions efficiently, ensuring smooth and secure operations. Implementing such robust backend processes in a School Management System can significantly enhance its performance and reliability, making it easier for schools to manage student and teacher data securely and efficiently[3].

Effective School Management

Everard, Morris, and Wilson (2004) provide a comprehensive guide on effective school management, covering aspects such as leadership, resource allocation, staff management, and policy implementation. They emphasize strategic planning and continuous improvement to achieve educational goals, discussing the role of technology in enhancing school management efficiency. Their work underscores the importance of having a well-planned and continuously evolving management strategy that integrates technology to streamline operations and improve educational outcomes. This approach can be directly applied to our School Management System, ensuring that it is not only a tool for day-to-day administration but also a strategic asset that supports long-term educational goals[4].

Classification of the Features in Learning Management Systems

Jurado et al. analyze the features of Learning Management Systems (LMS), classifying functionalities such as content delivery, communication tools, assessment mechanisms, and administrative capabilities. The study highlights how these features support instructional delivery, student engagement, and performance evaluation, underscoring the importance of selecting appropriate LMS features for specific educational needs. This analysis is crucial for our project as it provides a framework for identifying and integrating essential features into the School Management System. By ensuring that the system includes comprehensive administrative and educational functionalities, we can support both the management and instructional needs of schools[5].

Insights and Implementation

Acharya (2023, 2024b, 2024c) consistently emphasizes the role of centralized systems in improving data accuracy, streamlining administrative tasks, and enhancing overall efficiency. Everard, Morris, and Wilson (2004) focus on strategic planning and continuous improvement to enhance school management efficiency. Based on these insights, implementing a School Management System involves addressing both student and teacher management comprehensively.

Key functionalities include user authentication, administrative dashboards, and efficient management of student and teacher data. These tasks are critical for school management, aiming to enhance efficiency and support educational objectives. Additionally, the system should be user-friendly and accessible, balancing functionality with affordability, unifying various administrative tasks into a single system, and protecting sensitive information to ensure data integrity.

In conclusion, the literature highlights the importance of integrating technology into school management to improve efficiency, accuracy, and communication. By drawing on the strengths of existing systems and addressing their weaknesses, our School Management System can offer a comprehensive solution tailored to the needs of Sri Lankan schools. This system will not only streamline administrative tasks but also support strategic planning and continuous improvement, ultimately enhancing the educational experience for students and teachers alike.

3. Systems Analysis

3.1. System Requirements

1. Hardware Requirements:

Client Device:

- Desktop or laptop with internet access.
- Minimum 2GB RAM, 500MB free storage space.

2. Software Requirements:

Client:

- Web browser (Chrome, Firefox, Safari, Edge).
- Operating System: Windows, macOS, or Linux.

3. Security Requirements:

- User authentication and authorization.
- Data encryption (SSL/TLS for data in transit, AES for data at rest).
- Regular backups and secure storage of backup data.

3.2 Functional Requirements

Implementing a Login Page for Authentication:

- Implement a login system with robust authentication mechanisms, such as username and password.
- Define and manage user roles (e.g., administrator, teacher, student) with appropriate access permissions.

Creating a Dashboard for Administrators:

- Design a user-friendly dashboard for administrators to access various functionalities.
- Display counts of current students and teachers, as well as today's enrollment details.
- Provide easy access to student and teacher forms for data management.

Adding Functionality to Manage Student Records:

- Develop forms for inputting comprehensive student information (e.g., name, address, contact details).
- Ensure student records are securely stored in a database, with capabilities to update, clear, and delete data as needed.

Adding Functionality to Manage Teacher Records:

- Create forms to input detailed teacher information (e.g., name, address, contact details).
- Store teacher records securely in a database, and provide options to update, clear, and delete data as necessary.

3.3 Non-Functional Requirements

Performance Requirements:

- The system should respond to user interactions, such as login attempts, within acceptable time limits.
- It should handle a certain number of concurrent users without experiencing performance degradation.

Usability Requirements:

- The user interface should be intuitive and user-friendly, requiring minimal training for users to navigate and perform tasks.
- Clear and concise error messages should be provided to guide users in case of mistakes or issues.
- The application should support customization and personalization options to adapt to individual user preferences.

Reliability Requirements:

- The system should be highly reliable, with minimal downtime or crashes.
- It should accurately record and store student and teacher data without loss or corruption.
- Error handling mechanisms should be in place to gracefully handle unexpected errors and prevent data loss.

Security Requirements:

- Implement strong authentication mechanisms to ensure that only authorized users can access sensitive data.
- Enforce access controls to restrict user actions based on roles and permissions.
- Use data encryption to protect sensitive information stored in the system.

Scalability Requirements:

- The system should be scalable to accommodate a growing number of users and records.
- It should support horizontal scaling by adding more servers or instances to effectively distribute the workload.

Maintainability Requirements:

- The codebase should be well-structured, modular, and documented to facilitate easy maintenance and future enhancements.
- Version control systems should be used to manage code changes and track revisions.
- Regular updates and patches should be applied to address bugs, security vulnerabilities, and performance issues.

Compatibility Requirements:

- The application should be compatible with various operating systems and web browsers.
- It should support integration with other software tools or platforms commonly used in educational institutions.
- Compatibility with different screen resolutions and device types should be ensured for a seamless user experience.

3.4Summary

This chapter has outlined the comprehensive requirements for the School Management System (SMS). The system is designed to help educational institutions manage their administrative tasks, student and teacher records, and communication processes efficiently.

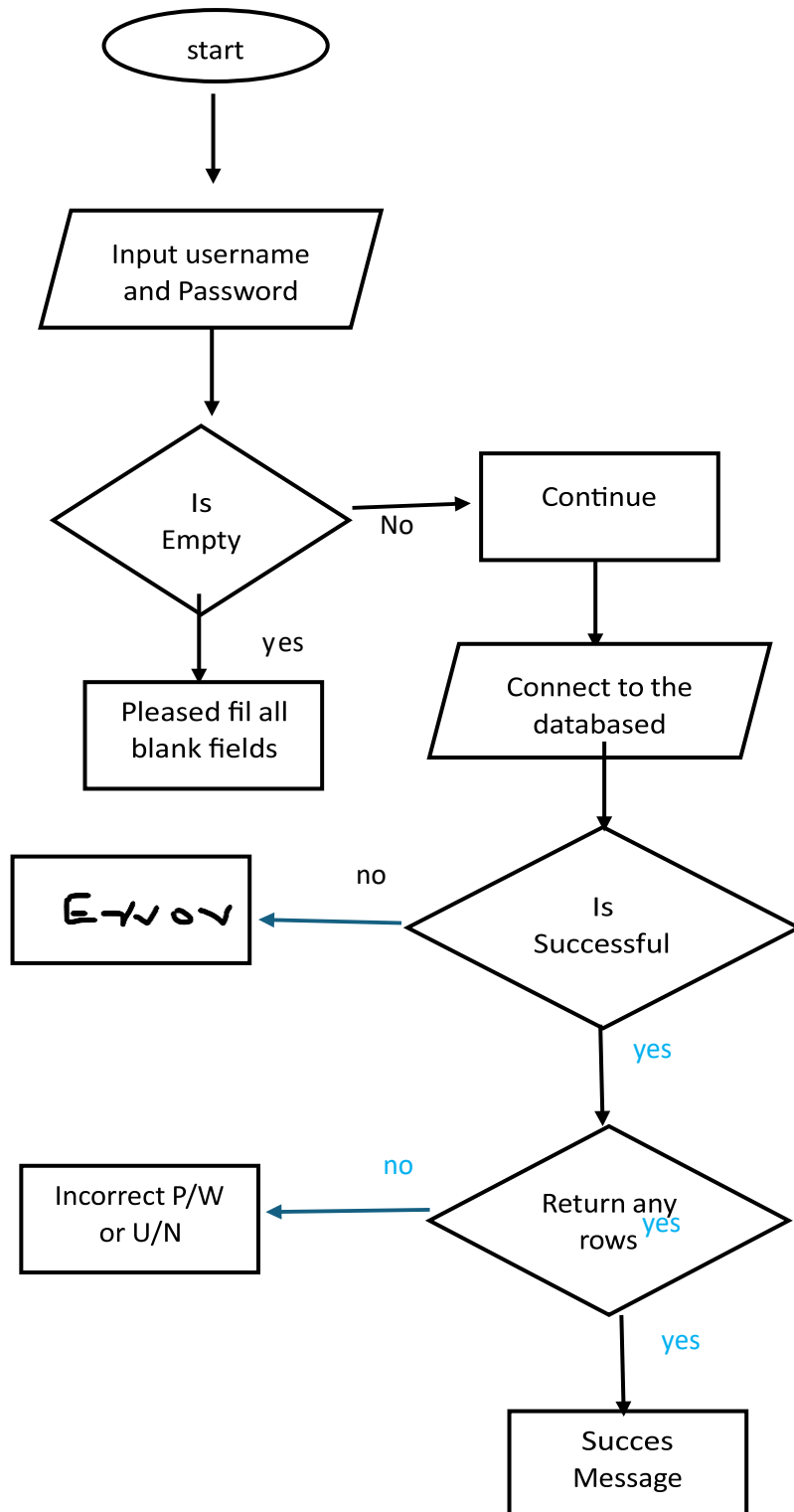
The hardware and software requirements ensure that both the client and server environments are suitable for running the system effectively, with a strong emphasis on security and data protection. The functional requirements cover the core functionalities of the system, including user authentication, dashboard access, and management of student and teacher records. These functionalities ensure that administrators can manage their tasks in a structured and user-friendly manner.

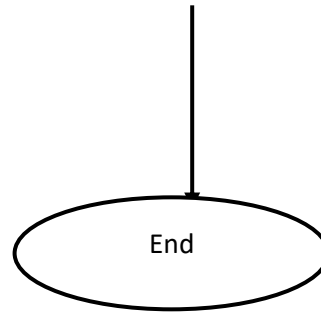
The non-functional requirements address the performance, reliability, usability, scalability, maintainability, and compatibility aspects of the system. These requirements ensure that the system not only meets the functional needs of the users but also provides a robust, efficient, and secure platform.

By addressing these detailed requirements, the School Management System aims to provide a comprehensive solution for managing school operations, ensuring a smooth and reliable user experience.

4. System Design

4.1 Login page





pseudo code

function login()

 if username or password is empty:

 display "Please fill all blank fields" error message

 else:

 try:

 connect to database

 execute SQL query to select user data

 if query successful:

 if query returned any rows:

 display "Login Successfully!" message

 proceed to main form

 else:

 display "Incorrect Username/Password" error message

 else:

 display "Error connecting Database" error message

 catch exception:

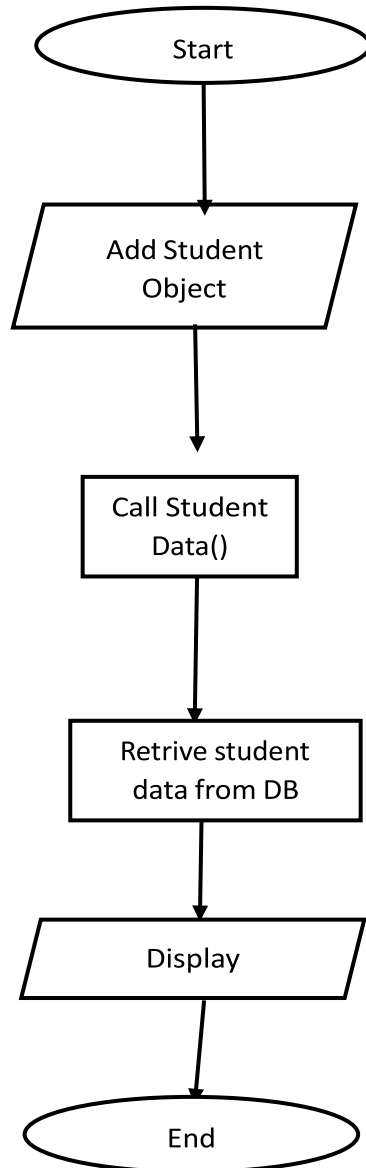
 display "Error connecting Database" error message

 finally:

 close database connection

4.2 Add student Data

Display Student Data



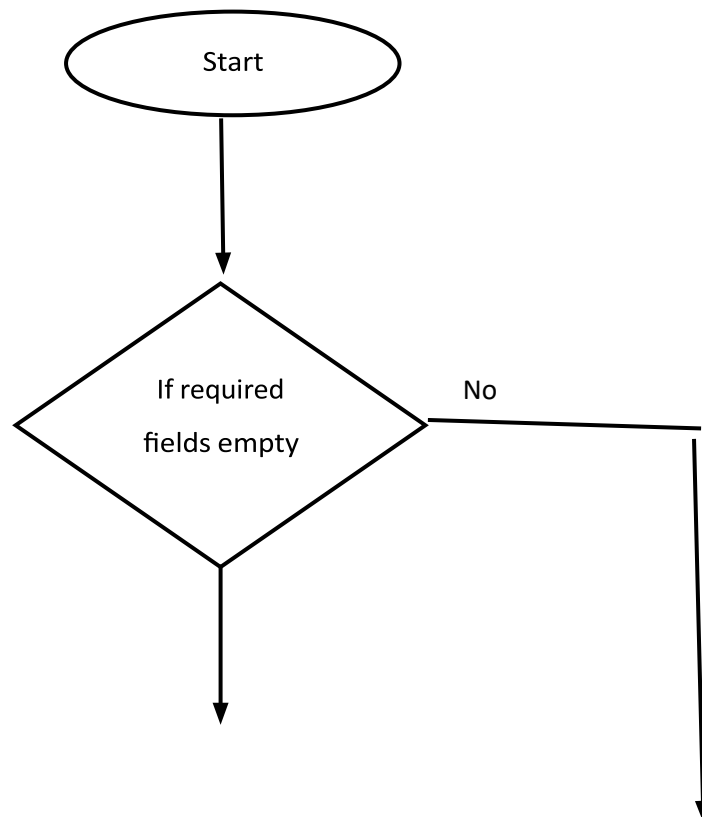
Function displayStudentData()

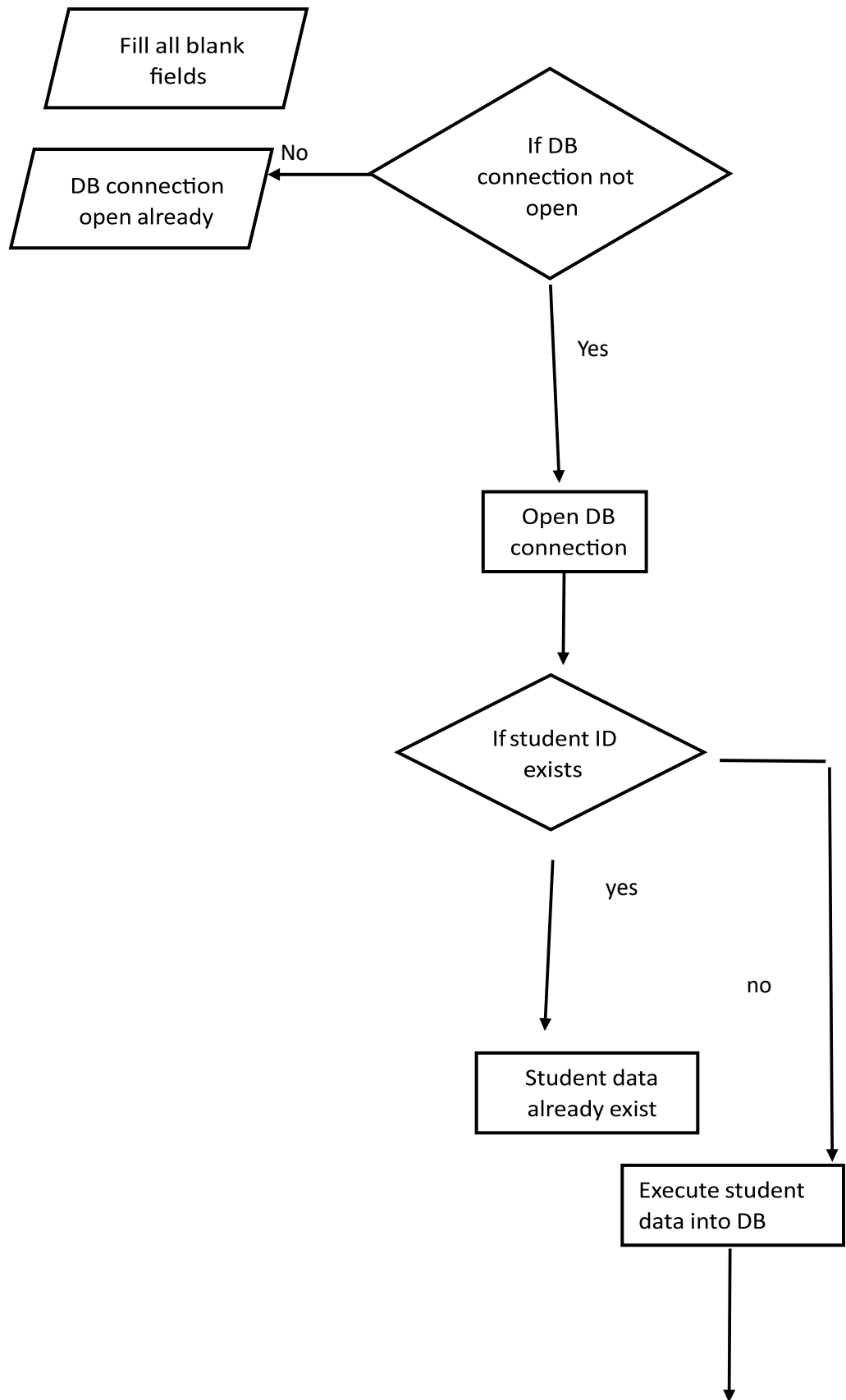
 instantiate AddStudentData object

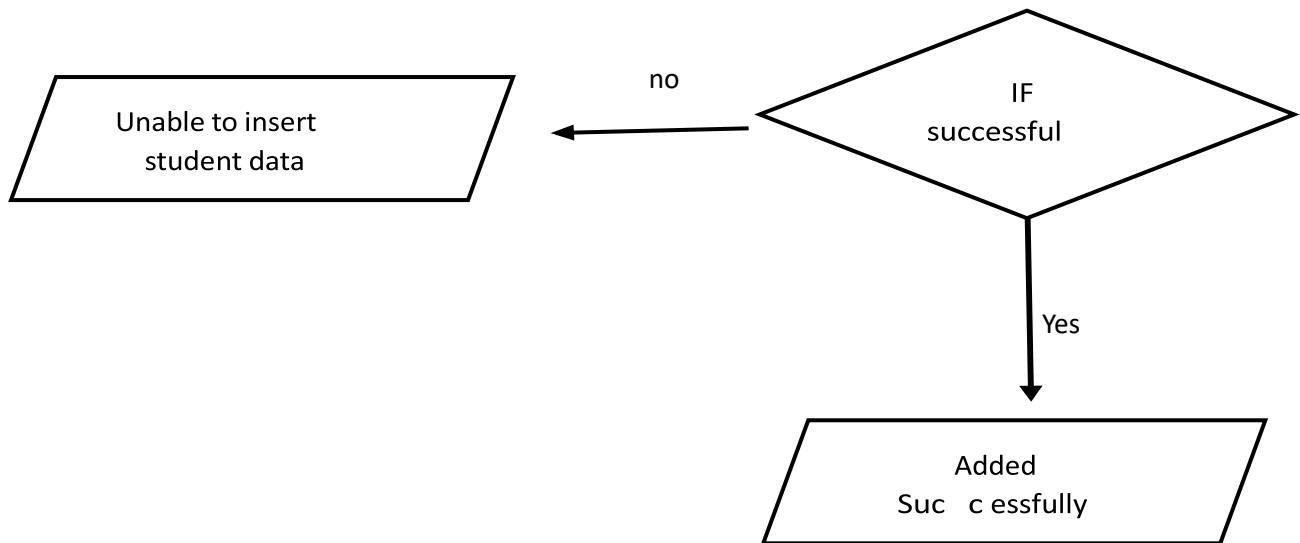
 retrieve student data from the database using studentData() method of AddStudentData object

 set the retrieved student data as the data source for the student_studentData control

Add Student data



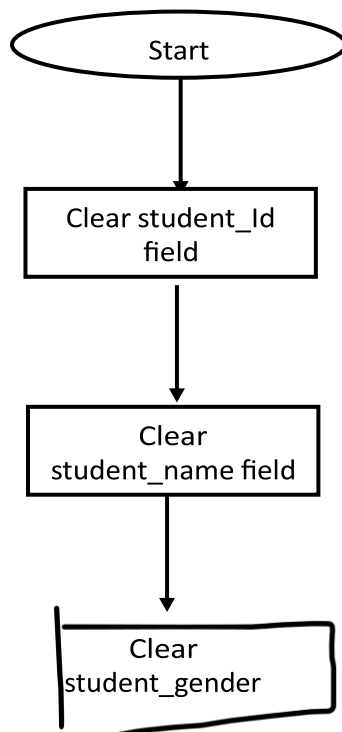


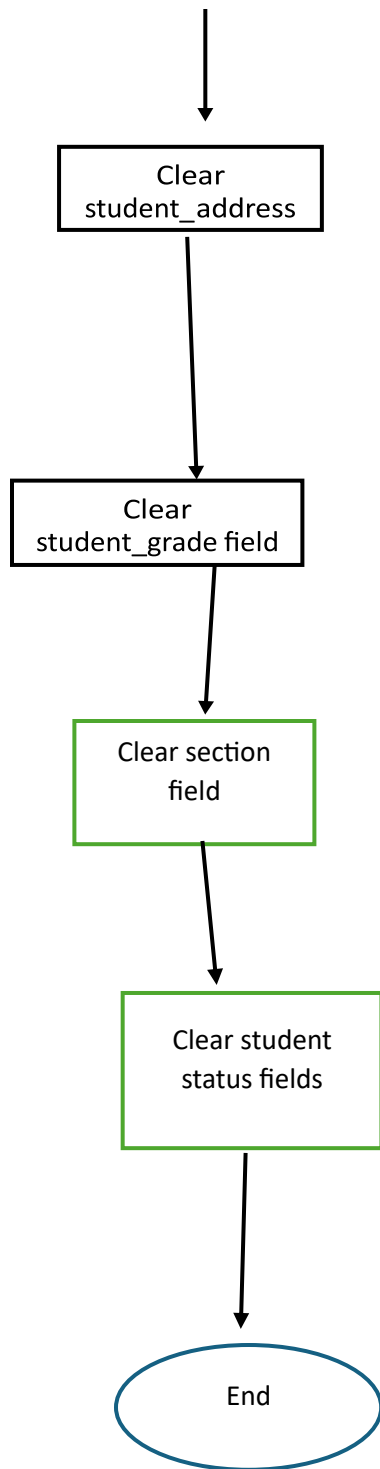


```
function addStudentData()  
  if any required field is empty:  
    display "Please fill all blank fields" error message  
  else if database connection is not open:  
    try:  
      open database connection  
      check if student ID already exists in the database  
      if student ID does not exist:  
        generate file path for saving student image  
      IF  
        Unable to insert successful  
        student data  
      Added  
      Successfully  
    16  
    create directory if it doesn't exist
```

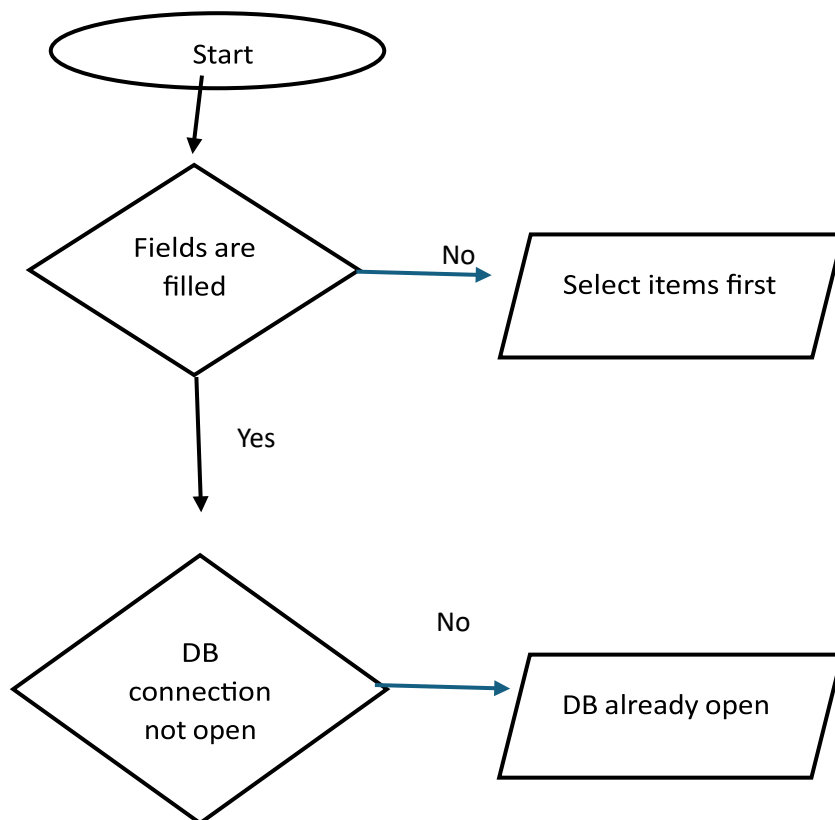
copy student image to the specified directory
execute SQL query to insert student data into the database
if insertion successful:
display "Added successfully!" success message
clear input fields
else:
display "Error: Unable to insert student data" error message
catch exception:
display "Error: Unable to connect to the database" error message
finally:
close database connection
else:
display "Error: Database connection is already open" error message

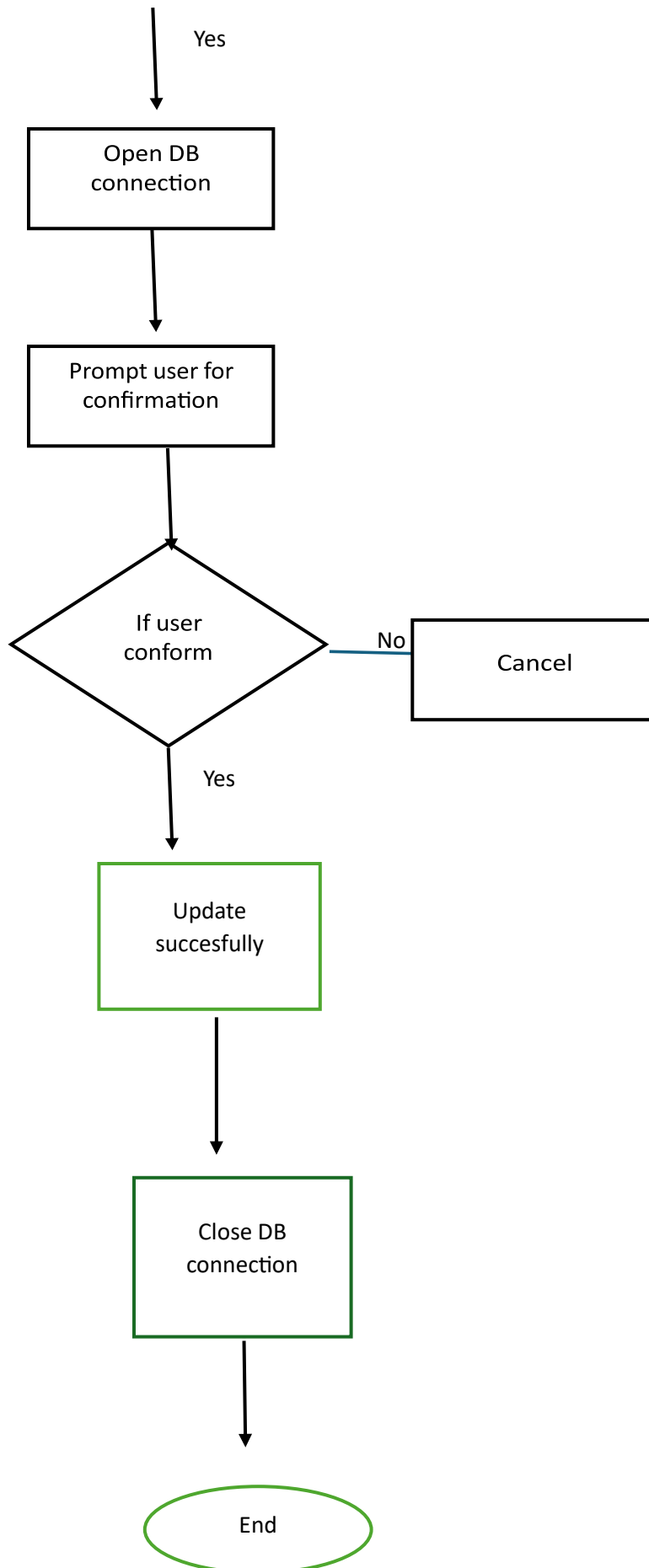
Clear Student data





```
function clearFields()  
  clear student_id field  
  clear student_name field  
  clear student_gender field  
  clear student_address field  
  clear student_grade field  
  clear student_section field  
  clear student_status field  
  clear student_image field  
  reset imagePath variable to empty string
```





Pseudo code

```
function student_updateBtn_Click()
    if any required fields are empty:
        display "Please select item first" error message
    else if database connection is not open:
        try:
            open database connection
            prompt user for confirmation to update student data
            if user confirms:
                get current date
                execute SQL query to update student data in the database
                display "Updated successfully!" message
                clear input fields
                refresh displayed student data
            else:
                display "Cancelled." message
        catch exception:
            display "Error connecting Database" error message
    finally:
        close database connection
    else:
        display "Error: Database connection is already open" error messag
```

5. System Implementation

5.1. Overview

This section details the implementation of the School Management System (SMS), which includes four primary user interfaces: Login Page, Administrator Dashboard, Student Management, and Teacher Management. The implementation covers the design, relevant inputs, generated outputs, and major processes to ensure a seamless and efficient management experience for educational institutions.

5.2. Login Page

The Login Page serves as the gateway to the SMS, providing secure access to the system. Users must authenticate themselves using their username and password. Based on their roles (administrator, teacher, or student), users are granted appropriate access permissions to various functionalities within the system.

```
namespace School_Management_System
```

```
{
```

```
    public partial class Loginform : Form
```

```
    {
```

```
        SqlConnection connect = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Asus\Documents\NewSchool
.mdf;Integrated Security=True;Connect Timeout=30");
```

```
        public Loginform()
```

```
        {
```

```
            InitializeComponent();
```

```
        }
```

```
        private void panel1_Paint(object sender, PaintEventArgs e)
```

```
        {
```

```

    }

    private void textBox1_TextChanged(object sender, EventArgs e)
    {

    }

    private void label1_Click(object sender, EventArgs e)
    {
        Application.Exit();
    }

    private void loginBtn_Click(object sender, EventArgs e)
    {
        if (username.Text == "" || password.Text == "")
        {
            MessageBox.Show("Please fill all blank fields", "Error Message",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else
        {

            try
            {
                connect.Open();

                String selectData = "SELECT * FROM users WHERE username = @username
AND password = @password";

```



```

using (SqlCommand cmd = new SqlCommand(selectData, connect))
{
    cmd.Parameters.AddWithValue("@username", username.Text.Trim());
    cmd.Parameters.AddWithValue("@password", password.Text.Trim());
    SqlDataAdapter adapter = new SqlDataAdapter(cmd);
    DataTable table = new DataTable();
    adapter.Fill(table);

    if (table.Rows.Count >= 1)
    {

        MessageBox.Show("Login Successfully!", "Information Message",
        MessageBoxButtons.OK, MessageBoxIcon.Information);

        MainForm mForm = new MainForm();
        mForm.Show();
        this.Hide();
    }
    else
    {
        MessageBox.Show("Incorrect Username/Password", "Error Message",
        MessageBoxButtons.OK, MessageBoxIcon.Error);

    }
}
catch (Exception ex)
{

```

```

        MessageBox.Show("Error connecting Database: " + ex, "Error Message",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally
    {
        connect.Close();
    }
}

}

private void showpass_CheckedChanged(object sender, EventArgs e)
{
    password.PasswordChar = showpass.Checked ? '\0' : '*';
}
}
}

```

5.3. Administrator Dashboard

The Administrator Dashboard is the central hub for administrators to manage various functionalities within the SMS. It provides an overview of current students and teachers, today's enrollment details, and access to student and teacher management forms.

```
namespace School_Management_System
```

```
{
```

```
    public partial class DashbordForm : UserControl
```

```
    {
```

```
        SqlConnection connect = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Asus\Documents\NewSchool
.mdf;Integrated Security=True;Connect Timeout=30");
```

```
        public DashbordForm()
```

```
        {
```

```
            InitializeComponent();
```

```
            displayTotalES();
```

```
            displayTotalGS();
```

```
            displayTotalTT();
```

```
            displayEnrolledStudentToday();
```

```
        }
```

```
        public void displayTotalES()
```

```
        {
```

```
            if (connect.State != ConnectionState.Open)
```

```
            {
```

```
                try
```

```
                {
```

```
                    connect.Open();
```

```
        string selectData = "SELECT COUNT(id) FROM students WHERE student_status  
= 'Enrolled' AND date_delete IS NULL";
```

```
        using (SqlCommand cmd = new SqlCommand(selectData, connect))  
        {  
            SqlDataReader reader = cmd.ExecuteReader();  
            int tempES = 0;  
            if (reader.Read())  
            {  
                tempES = Convert.ToInt32(reader[0]);  
  
                total_ES.Text = tempES.ToString();  
            }  
        }  
  
    }  
    catch (Exception ex)  
    {  
        MessageBox.Show("Error to connect Database: " + ex, "Error Message",  
        MessageBoxButtons.OK, MessageBoxIcon.Error);  
  
    }  
    finally  
    {  
        connect.Close();  
    }  
}  
}
```

```

public void displayTotalTT()
{
    if (connect.State != ConnectionState.Open)
    {
        try
        {
            connect.Open();

            string selectData = "SELECT COUNT(id) FROM teachers WHERE teacher_status
= 'Active' AND date_delete IS NULL";

            using (SqlCommand cmd = new SqlCommand(selectData, connect))
            {
                SqlDataReader reader = cmd.ExecuteReader();
                int tempTT = 0;
                if (reader.Read())
                {
                    tempTT = Convert.ToInt32(reader[0]);

                    total_TT.Text = tempTT.ToString();
                }
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show("Error to connect Database: " + ex, "Error Message",
MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
    }
}

```

```

        finally
        {
            connect.Close();
        }
    }
}

public void displayTotalGS()
{
    if (connect.State != ConnectionState.Open)
    {
        try
        {
            connect.Open();

            string selectData = "SELECT COUNT(id) FROM students WHERE student_status
= 'Graduated' AND date_delete IS NULL";

            using (SqlCommand cmd = new SqlCommand(selectData, connect))
            {
                SqlDataReader reader = cmd.ExecuteReader();
                int tempGS = 0;
                if (reader.Read())
                {
                    tempGS = Convert.ToInt32(reader[0]);

                    total_GS.Text = tempGS.ToString();
                }
            }
        }
    }
}

```

```

    }
    catch (Exception ex)
    {
        MessageBox.Show("Error to connect Database: " + ex, "Error Message",
        MessageBoxButtons.OK, MessageBoxIcon.Error);

    }
    finally
    {
        connect.Close();
    }
}

public void displayEnrolledStudentToday()
{
    AddStudentData asData = new AddStudentData();

    dataGridView1.DataSource = asData.dashboardStudentData();
}

private void pictureBox1_Click(object sender, EventArgs e)
{
}

private void DashbordForm_Load(object sender, EventArgs e)
{
}

```

```
}  
}
```

5.4. Student Management

The Student Management interface allows administrators to add, update, clear, and delete student records. This interface ensures that all student information is accurately captured and stored in the database.

```
namespace School_Management_System
```

```
{
```

```
    internal class AddStudentData
```

```
    {
```

```
        SqlConnection connect = new SqlConnection(@"Data  
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Asus\Documents\NewSchool  
.mdf;Integrated Security=True;Connect Timeout=30");
```

```
        public int ID { set; get; }
```

```
        public string StudentID { set; get; }
```

```
        public string StudentName { set; get; }
```

```
        public string StudentGender { set; get; }
```

```
        public string StudentAddress { set; get; }
```

```
        public string StudentGrade { set; get; }
```

```
        public string StudentSection { set; get; }
```

```
        public string StudentImage { set; get; }
```

```
        public string Status { set; get; }
```

```
        public string DateInsert { set; get; }
```

```
        public List<AddStudentData> studentData()
```

```
        {
```

```
            List<AddStudentData> listData = new List<AddStudentData>();
```



```

if (connect.State != ConnectionState.Open)
{
    try
    {
        connect.Open();

        string sql = "SELECT * FROM students WHERE date_delete IS NULL";

        using (SqlCommand cmd = new SqlCommand(sql, connect))
        {
            SqlDataReader reader = cmd.ExecuteReader();

            while (reader.Read())
            {
                AddStudentData addSD = new AddStudentData();
                addSD.ID = (int)reader["id"];
                addSD.StudentID = reader["student_id"].ToString();
                addSD.StudentName = reader["student_name"].ToString();
                addSD.StudentGender = reader["student_gender"].ToString();
                addSD.StudentAddress = reader["student_address"].ToString();
                addSD.StudentGrade = reader["student_grade"].ToString();
                addSD.StudentSection = reader["student_section"].ToString();
                addSD.StudentImage = reader["student_image"].ToString();
                addSD.Status = reader["student_status"].ToString();
                addSD.DateInsert = reader["date_insert"].ToString();

                listData.Add(addSD);
            }
        }
    }
}

```

```

        reader.Close();
    }
}
catch (Exception ex)
{
    Console.WriteLine("Error connecting Database: " + ex);
}
finally
{
    connect.Close();
}
}
return listData;
}

```

```

public List<AddStudentData> dashboardStudentData()
{
    List<AddStudentData> listData = new List<AddStudentData>();

    if (connect.State != ConnectionState.Open)
    {
        try
        {
            connect.Open();
            DateTime today = DateTime.Today;

```

```

string sql = "SELECT * FROM students WHERE date_insert = @dateInsert " +
    "AND date_delete IS NULL";

using (SqlCommand cmd = new SqlCommand(sql, connect))
{
    cmd.Parameters.AddWithValue("@dateInsert", today.ToString());
    SqlDataReader reader = cmd.ExecuteReader();

    while (reader.Read())
    {
        AddStudentData addSD = new AddStudentData();
        addSD.ID = (int)reader["id"];
        addSD.StudentID = reader["student_id"].ToString();
        addSD.StudentName = reader["student_name"].ToString();
        addSD.StudentGender = reader["student_gender"].ToString();
        addSD.StudentAddress = reader["student_address"].ToString();
        addSD.StudentGrade = reader["student_grade"].ToString();
        addSD.StudentSection = reader["student_section"].ToString();
        addSD.StudentImage = reader["student_image"].ToString();
        addSD.Status = reader["student_status"].ToString();
        addSD.DateInsert = reader["date_insert"].ToString();

        listData.Add(addSD);
    }
    reader.Close();
}
}
catch (Exception ex)

```

```

        {
            Console.WriteLine("Error: " + ex);
        }
        finally
        {
            connect.Close();
        }
    }
    return listData;
}
}
}

```

5.5. Teacher Management

The Teacher Management interface offers similar functionalities to the Student Management interface, allowing administrators to add, update, clear, and delete teacher records. This interface ensures that all teacher information is accurately maintained in the database.

```

namespace School_Management_System
{
    public partial class AddTeacherForm : UserControl
    {
        SqlConnection connect = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Asus\Documents\NewSchool
.mdf;Integrated Security=True;Connect Timeout=30");

        public AddTeacherForm()
        {
            InitializeComponent();
        }
    }
}

```

```

        teacherDisplayData();
    }

    public void teacherDisplayData()
    {
        AddTeacherData addTD = new AddTeacherData();

        teacher_gridData.DataSource = addTD.teacherData();
    }

    private void teacher_addBtn_Click(object sender, EventArgs e)
    {
        if (teacher_id.Text == ""
            || teacher_name.Text == ""
            || teacher_gender.Text == ""
            || teacher_address.Text == ""
            || teacher_status.Text == ""
            || teacher_image.Image == null
            || imagePath == null)
        {
            MessageBox.Show("Please fill all blank fields", "Error Message",
                MessageBoxButtons.OK, MessageBoxIcon.Error);
        }
        else
        {
            if (connect.State != ConnectionState.Open)

```

```

{
    try
    {
        connect.Open();

        // WE DON'T LIKE THE DUPLICATE TEACHER ID SO, WE NEED TO
        CHECK IF ON THE DATABASE HAS ALREADY TEACHER ID VALUE THAT SAME TO
        THE USER THAT WANT TO INSERT

        string checkTeacherID = "SELECT COUNT(*) FROM teachers WHERE
teacher_id = @teacherID";

        using (SqlCommand checkTID = new SqlCommand(checkTeacherID, connect))
        {
            checkTID.Parameters.AddWithValue("@teacherID", teacher_id.Text.Trim());
            int count = (int)checkTID.ExecuteScalar();

            if (count >= 1)
            {
                MessageBox.Show("Teacher ID: " + teacher_id.Text.Trim() + " is already
exist"
                , "Error Message", MessageBoxButtons.OK, MessageBoxIcon.Error);
            }
            else
            {
                DateTime today = DateTime.Today;
                string insertData = "INSERT INTO teachers " +
                "(teacher_id,teacher_name,teacher_geder, teacher_address, " +
                "teacher_image, teacher_status, date_insert) " +

```

```

VALUES(@teacherID, @teacherName, @teacherGender,
@teacherAddress," +
"@teacherImage, @teacherStatus, @dateInsert)";

// TO SAVE TO YOUR DIRECTORY

string path =
Path.Combine(@"C:\Users\Asus\Desktop\Project\School_Management_System\School_Manage
ment_System\Teacher_Directory\", teacher_id.Text.Trim() + ".jpg");

string directoryPath = Path.GetDirectoryName(path);

if (!Directory.Exists(directoryPath))
{
    Directory.CreateDirectory(directoryPath);
}

File.Copy(imagePath, path, true);

using (SqlCommand cmd = new SqlCommand(insertData, connect))
{
    cmd.Parameters.AddWithValue("@teacherID", teacher_id.Text.Trim());
    cmd.Parameters.AddWithValue("@teacherName",
teacher_name.Text.Trim());
    cmd.Parameters.AddWithValue("@teacherGender",
teacher_gender.Text.Trim());
    cmd.Parameters.AddWithValue("@teacherAddress",
teacher_address.Text.Trim());
    cmd.Parameters.AddWithValue("@teacherImage", path);

```

```

        cmd.Parameters.AddWithValue("@teacherStatus",
teacher_status.Text.Trim());

        cmd.Parameters.AddWithValue("@dateInsert", today.ToString());

        cmd.ExecuteNonQuery();

        teacherDisplayData();

        MessageBox.Show("Added successfully!", "Information Message",
MessageBoxButtons.OK, MessageBoxIcon.Information);

        clearFields();
    }
}
}
}
catch (Exception ex)
{
    MessageBox.Show("Error connecting Database: " + ex, "Error Message",
MessageBoxButtons.OK, MessageBoxIcon.Error);

}
finally
{
    connect.Close();
}
}

```



```

    }
}

public void clearFields()
{
    teacher_id.Text = "";
    teacher_name.Text = "";
    teacher_gender.SelectedIndex = -1;
    teacher_address.Text = "";
    teacher_status.SelectedIndex = -1;
    teacher_image.Image = null;
    imagePath = "";
}

private string imagePath;
private void button1_Click(object sender, EventArgs e)
{
    OpenFileDialog open = new OpenFileDialog();
    open.Filter = "Image files (*.jpg; *.png)|*.jpg;*.png";

    if (open.ShowDialog() == DialogResult.OK)
    {
        imagePath = open.FileName;

        teacher_image.ImageLocation = imagePath;
    }
}

```

```

private void teacher_clearBtn_Click(object sender, EventArgs e)
{
    clearFields();
}

private void teacher_updateBtn_Click(object sender, EventArgs e)
{
    if (teacher_id.Text == ""
        || teacher_name.Text == ""
        || teacher_gender.Text == ""
        || teacher_address.Text == ""
        || teacher_status.Text == ""
        || teacher_image.Image == null
        || imagePath == null)
    {
        MessageBox.Show("Please select item first", "Error Message",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else
    {
        if (connect.State != ConnectionState.Open)
        {
            try
            {
                connect.Open();

                DialogResult check = MessageBox.Show("Are you sure you want to Update
Teacher ID: "

```

```
        + teacher_id.Text.Trim() + "?", "Confirmation Message",  
        MessageBoxButtons.YesNo, MessageBoxIcon.Question);
```

```
    if (check == DialogResult.Yes)
```

```
    {
```

```
        DateTime today = DateTime.Today;
```

```
        String updateData = "UPDATE teachers SET " +
```

```
            "teacher_name = @teacherName, teacher_geder = @teacherGender" +
```

```
            ", teacher_address = @teacherAddress" +
```

```
            ", teacher_status = @teacherStatus" +
```

```
            ", date_update = @dateUpdate WHERE teacher_id = @teacherID";
```

```
        using (SqlCommand cmd = new SqlCommand(updateData, connect))
```

```
        {
```

```
            cmd.Parameters.AddWithValue("@teacherName",  
teacher_name.Text.Trim());
```

```
            cmd.Parameters.AddWithValue("@teacherGender",  
teacher_gender.Text.Trim());
```

```
            cmd.Parameters.AddWithValue("@teacherAddress",  
teacher_address.Text.Trim());
```

```
            cmd.Parameters.AddWithValue("@teacherStatus",  
teacher_status.Text.Trim());
```

```
            cmd.Parameters.AddWithValue("@dateUpdate", today.ToString());
```

```
            cmd.Parameters.AddWithValue("@teacherID", teacher_id.Text.Trim());
```

```
            cmd.ExecuteNonQuery();
```

```
            teacherDisplayData();
```

```

        MessageBox.Show("Updated successfully!", "Information Message",
        MessageBoxButtons.OK, MessageBoxIcon.Information);

        clearFields();

    }
}
else
{
    MessageBox.Show("Cancelled.", "Information Message",
    MessageBoxButtons.OK, MessageBoxIcon.Error);
    clearFields();
}
}
catch (Exception ex)
{
    MessageBox.Show("Error connecting Database: " + ex, "Error Message",
    MessageBoxButtons.OK, MessageBoxIcon.Error);

}
finally
{
    connect.Close();
}
}
}
}

```

```

private void teacher_gridData_CellClick(object sender, DataGridViewCellEventArgs e)
{
    if (e.RowIndex != -1)
    {
        DataGridViewRow row = teacher_gridData.Rows[e.RowIndex];
        teacher_id.Text = row.Cells[1].Value.ToString();
        teacher_name.Text = row.Cells[2].Value.ToString();
        teacher_gender.Text = row.Cells[3].Value.ToString();
        teacher_address.Text = row.Cells[4].Value.ToString();

        imagePath = row.Cells[5].Value.ToString();

        string imageData = row.Cells[5].Value.ToString();

        if (imageData != null && imageData.Length > 0)
        {
            teacher_image.Image = Image.FromFile(imageData);
        }
        else
        {
            teacher_image.Image = null;
        }

        teacher_status.Text = row.Cells[6].Value.ToString();

    }
}

```

```

private void teacher_deleteBtn_Click(object sender, EventArgs e)
{
    if (teacher_id.Text == "")
    {
        MessageBox.Show("Please select item first", "Error Message"
            , MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    else
    {
        if (connect.State != ConnectionState.Open)
        {
            DialogResult check = MessageBox.Show("Are you sure you want to Delete Teacher
ID: "
            + teacher_id.Text + "?", "Confirmation Message", MessageBoxButtons.YesNo,
            MessageBoxIcon.Question);

            if (check == DialogResult.Yes)
            {
                try
                {
                    connect.Open();
                    DateTime today = DateTime.Today;

                    string deleteData = "UPDATE teachers SET date_delete = @dateDelete " +
                        "WHERE teacher_id = @teacherID";

                    using (SqlCommand cmd = new SqlCommand(deleteData, connect))

```

```

        {
            cmd.Parameters.AddWithValue("@dateDelete", today.ToString());
            cmd.Parameters.AddWithValue("@teacherID", teacher_id.Text.Trim());

            cmd.ExecuteNonQuery();

            teacherDisplayData();

            MessageBox.Show("Deleted successfully!", "Information Message",
                MessageBoxButtons.OK, MessageBoxIcon.Information);

            clearFields();
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show("Error connecting Database: " + ex, "Error Message",
            MessageBoxButtons.OK, MessageBoxIcon.Error);
    }
    finally
    {
        connect.Close();
    }
}
else
{
    MessageBox.Show("Cancelled.", "Information Message",
        MessageBoxButtons.OK, MessageBoxIcon.Error);
}
}

```

```
    }  
  }  
}  
}
```

5.6. Appendix

Database Schema :

MySQL

CREATE TABLE users

```
(  
  id INT PRIMARY KEY IDENTITY (1,1),  
  username VARCHAR(MAX) NULL,  
  password VARCHAR(MAX) NULL  
)
```

SELECT * FROM users;

INSERT INTO users(username,password) VALUES('sayuu','sayuu123');

```
CREATE TABLE students(  
  id INT PRIMARY KEY IDENTITY(1,1),  
  student_id VARCHAR(MAX) NULL,  
  student_name VARCHAR(MAX) NULL,  
  student_gender VARCHAR(MAX) NULL,
```



```
student_address VARCHAR(MAX) NULL,  
student_grade VARCHAR(MAX) NULL,  
student_section VARCHAR(MAX) NULL,  
student_image VARCHAR(MAX) NULL,  
student_status VARCHAR(MAX) NULL,  
student_insert VARCHAR(MAX) NULL,  
date_insert DATE NULL,  
date_update DATE NULL,  
date_delete DATE NULL  
)
```

```
SELECT * FROM students;
```

```
CREATE TABLE teachers  
(  
id INT PRIMARY KEY IDENTITY(1,1),  
teacher_id VARCHAR(MAX) NULL,  
teacher_name VARCHAR(MAX) NULL,  
teacher_geder VARCHAR(MAX) NULL,  
teacher_address VARCHAR(MAX) NULL,  
teacher_image VARCHAR(MAX) NULL,  
teacher_status VARCHAR(MAX) NULL,  
date_insert DATE NULL,  
date_update DATE NULL,  
date_delete DATE NULL  
)
```

```
SELECT * FROM teachers;
```

```
SELECT * FROM students;
```

```
SELECT COUNT(id) FROM students WHERE student_status = 'Enrolled' AND date_delete IS  
NULL;
```

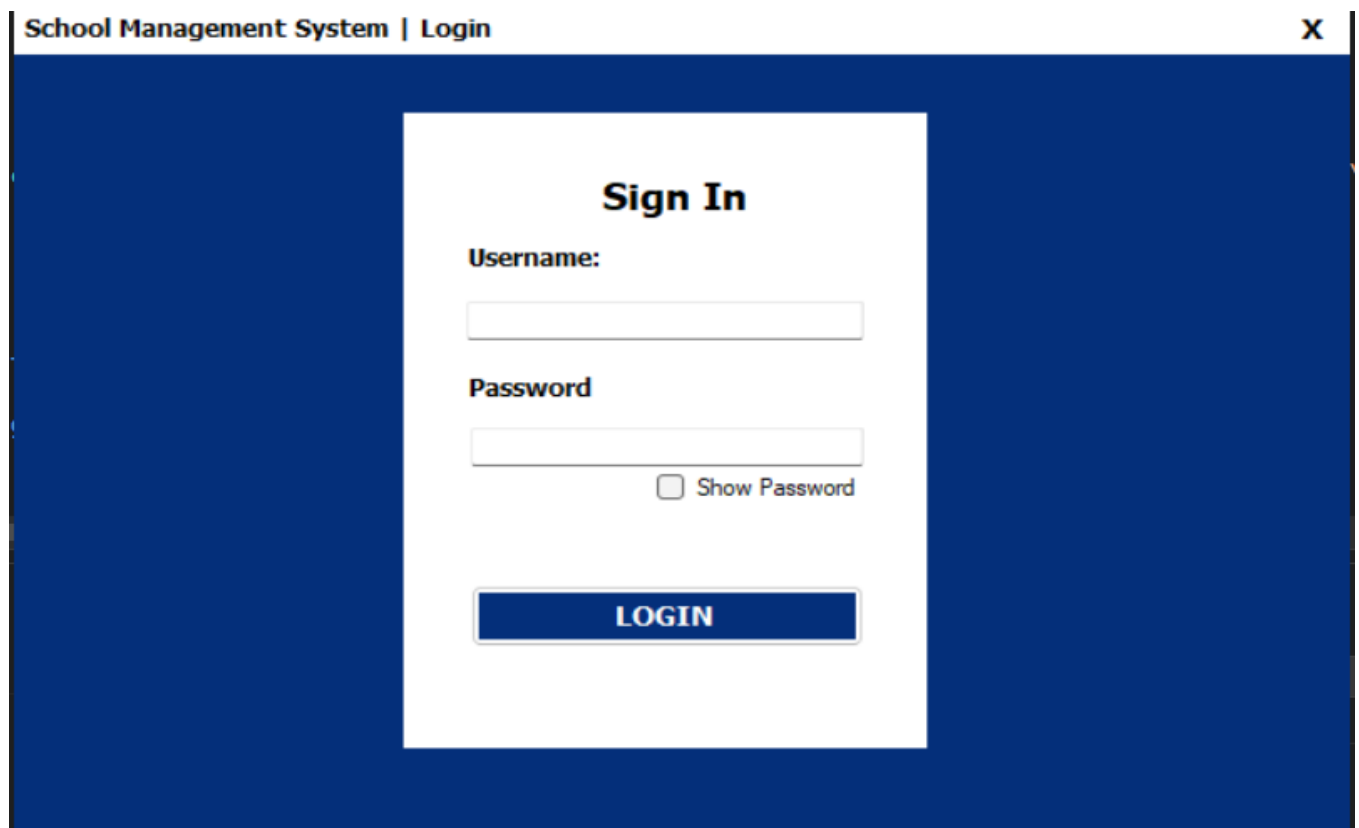
```
SELECT COUNT(id) FROM students WHERE student_status = 'Graduated' AND date_delete IS  
NULL;
```

6. Appendix

Appendix A:

User Interfaces

6.1login page



The screenshot shows a web browser window with the title "School Management System | Login" and a close button "X" in the top right corner. The main content area has a dark blue background. In the center, there is a white rectangular box containing the login form. The form is titled "Sign In" in bold black text. Below the title, there are two input fields: "Username:" and "Password". The "Password" field has a "Show Password" checkbox to its right. At the bottom of the form is a blue button with the text "LOGIN" in white capital letters.

6.1 login page

Buttons:

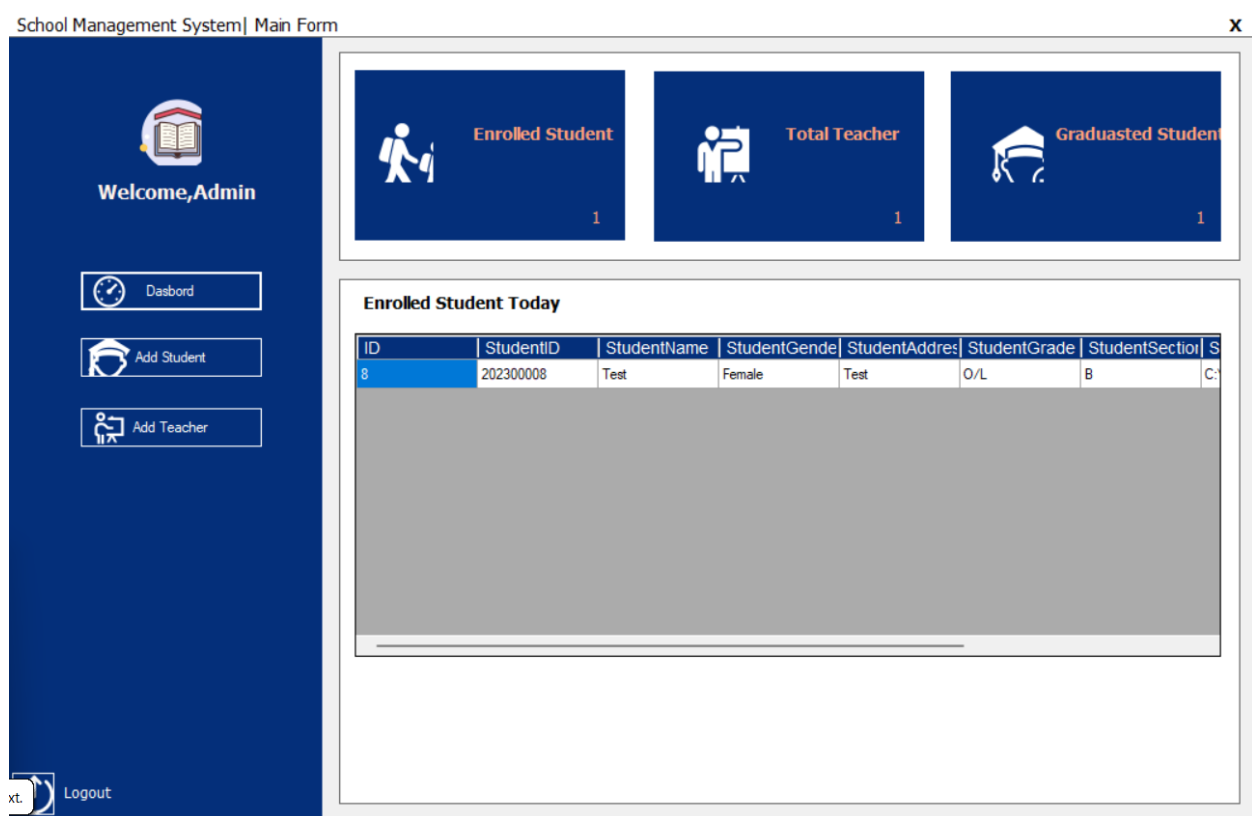
Login button: Submits the the user details.

Fields:

Username: add user name

Passwords: add user passwords

6.2Dashboard



6.3 Dashboard

Buttons:

Dashboard: go to dashboard screen.

Add student: add new student record

Add Teacher:add new teacher record

Delete: Delete new student Data

Import: Import student picture

Fields:

Student id: add student id

Student name: add student id

Address: add student Address

Status: add student status

Grade: add student grade

Gender: add student Gender

Section: add student section

6.5 Add Student

School Management System| Main Form X



Welcome, Admin

 Dashboard

 Add Student

 Add Teacher

 Logout

Teacher's Data

ID	TeacherID	TeacherName	TeacherGender	TeacherAddress	TeacherImage	Status	Date
1	TID-1	Test-Name1	Female	Test-Address	C:\Users\Asus\D...	Active	6/9/2
7	TID-9	Test	Male	Test	C:\Users\Asus\D...	Inactive	6/10

Teacher ID Address Status

Full Name

Gender

 Import

6.5 add teacher

Buttons:

Add: Add new teacher Data

Update: update new teacher Data

Clear: Clear new teacher Data

Delate: Delete new teacher Data

Import: Import teacher picture

Fields:

Student id: add teacher id

Student name: add teacher id

Address: add teacher Address

Status: add teacher status

Grade: add teacher grade

Gender: add teacher Gender

Section: add teacher section

Appendix B:

Code Snippets

MainForm

```
namespace School_Management_System
{
    public partial class MainForm : Form
    {
        public MainForm()
        {
            InitializeComponent();
        }
    }
}
```

```

    }

    private void label3_Click(object sender, EventArgs e)
    {
        Application.Exit();
    }

    private void button4_Click(object sender, EventArgs e)
    {
        DialogResult check = MessageBox.Show("Are you sure you want to logout?",
        "Confirmation Message", MessageBoxButtons.YesNo, MessageBoxIcon.Question);

        if (check == DialogResult.Yes)
        {
            Loginform lForm = new Loginform();
            lForm.Show();
            this.Hide();
        }
    }

    private void button3_Click(object sender, EventArgs e)
    {
        dashbordForm2.Visible = false;
        addStudentFrom2.Visible = false;
        addTeacherForm2.Visible = true;
    }

    //private void button1_Click(object sender, EventArgs e)
    //{

```



```

DashbordForm dForm = new DashbordForm();

private void button1_Click(object sender, EventArgs e)
{
    dashbordForm2.Visible = true;
    addStudentFrom2.Visible = false;
    addTeacherForm2.Visible = false;

}

private void addStudentBtn_Click(object sender, EventArgs e)
{
    dashbordForm2.Visible = false;
    addStudentFrom2.Visible = true;
    addTeacherForm2.Visible = false;

}
}
}

```

AddTeacherData

```

namespace School_Management_System
{
    internal class AddTeacherData
    {
        SqlConnection connect = new SqlConnection(@"Data
Source=(LocalDB)\MSSQLLocalDB;AttachDbFilename=C:\Users\Asus\Documents\NewSchool
.mdf;Integrated Security=True;Connect Timeout=30");

        public int ID { set; get; }
    }
}

```

```

public string TeacherID { set; get; }
public string TeacherName { set; get; }
public string TeacherGender { set; get; }
public string TeacherAddress { set; get; }
public string TeacherImage { set; get; }
public string Status { set; get; }
public string DateInsert { set; get; }

public List<AddTeacherData> teacherData()
{
    List<AddTeacherData> listData = new List<AddTeacherData>();
    if (connect.State != ConnectionState.Open)
    {
        try
        {
            connect.Open();

            string sql = "SELECT * FROM teachers WHERE date_delete IS NULL";

            using (SqlCommand cmd = new SqlCommand(sql, connect))
            {
                SqlDataReader reader = cmd.ExecuteReader();

                while (reader.Read())
                {
                    AddTeacherData addTD = new AddTeacherData();
                    addTD.ID = (int)reader["id"];
                    addTD.TeacherID = reader["teacher_id"].ToString();
                }
            }
        }
        catch { }
    }
}

```

```

        addTD.TeacherName = reader["teacher_name"].ToString();
        addTD.TeacherGender = reader["teacher_geder"].ToString();
        addTD.TeacherAddress = reader["teacher_address"].ToString();

        addTD.TeacherImage = reader["teacher_image"].ToString();
        addTD.Status = reader["teacher_status"].ToString();
        addTD.DateInsert = reader["date_insert"].ToString();

        listData.Add(addTD);
    }
    reader.Close();
}
}
catch (Exception ex)
{
    Console.WriteLine("Error connecting Database: " + ex);
}
finally
{
    connect.Close();
}
}
return listData;

}

```

6. References

- [1] Acharya, K. (2023). Teachers Record Management System Project Report (SSRN Scholarly Paper 4833821). <https://doi.org/10.2139/ssrn.4833821>
- [2] Acharya, K. (2024b). Online Bus Reservation System (SSRN Scholarly Paper 4807293). <https://doi.org/10.2139/ssrn.4807293>
- [3] Acharya, K. (2024c). Online shopping management system. (SSRN Scholarly Paper 4813594). <https://doi.org/10.2139/ssrn.4813594>
- [4] Everard, K. B., Morris, G., & Wilson, I. (2004). Effective School Management. SAGE.
- [5] Jurado, R. G., Pettersson, T., Gomez, A. R., & Scheja, M. (n.d.). CLASSIFICATION OF THE FEATURES IN LEARNING MANAGEMENT SYSTEMS.